

LeetCode Python Cheat Sheet

This sheet is for pattern recognition, not memorizing problem numbers. For quant and AI interviews, that usually means getting very fast at arrays, strings, heaps, binary search, graph traversal, optimization, and stream-style thinking.

The goal of each section is:

- recognize the pattern fast
- understand the invariant
- reuse a Python template you can explain out loud

Pattern Selection Flowchart

Read this top to bottom like a quick flowchart.

If you see this	Try this first	Quick cue
Sorted array	Binary Search, Two Pointers	Search a boundary or shrink from both ends
All permutations / subsets / combinations	Backtracking	Try a choice, recurse, undo
Tree	DFS, BFS	DFS for subtree/path logic, BFS for levels
Graph or grid	DFS, BFS, Dijkstra, Topological Sort, Union Find	Traverse, shortest path, dependencies, or connectivity
Linked list	Dummy Node, Fast/Slow Pointers, Reversal	Pointer wiring is the whole problem
Recursion is banned	Stack	Simulate recursion iteratively
Must solve in-place	Two Pointers, Swap-to-Index, Reverse Sections, Encode State In-Place	Reuse the array instead of extra memory
Maximum / minimum over many choices	DP, Greedy, Sliding Window, Binary Search on Answer	Optimize over states, choices, or feasible answers
Top / least K items	Heap, QuickSelect, Bucket Sort	Repeated best item vs one-shot kth
Common string / prefix search	HashMap, Trie	Exact lookup vs prefix lookup
Subarray / substring	Sliding Window, Prefix Sum + HashMap	Local valid window vs exact cumulative total
Else	HashMap / Set, then Sorting	Fast lookup or expose order

Quant / AI Focus

If your goal is quant or AI-heavy interview prep, spend extra time on these first:

Tier 1: Highest value

- HashMap / Set
- Sorting + Two Pointers
- Sliding Window
- Prefix Sum + HashMap
- Binary Search
- Heap / Priority Queue
- Graph / Grid BFS/DFS
- Topological Sort
- Dynamic Programming
- Greedy
- Design Problems

Tier 2: Important

- Stack
- Monotonic Stack
- Dijkstra
- Intervals
- QuickSelect
- Tree DFS / BFS

Tier 3: Usually lower frequency, but still interviewable

- Linked List
- Union Find
- Backtracking
- Trie
- In-Place Array Tricks

Why this ordering:

- Quant interviews often over-index on arrays, time series, ranking, optimization, and fast reasoning on numeric data.
- AI / ML / infra interviews often over-index on search, top-k, graphs, pipelines, caching, retrieval, and stream-style processing.

Python Template Habits

- Use short names with meaning: `left`, `right`, `ans`, `freq`, `q`, `heap`, `dist`, `parent`.
- Put the invariant in a comment if the loop is subtle.
- Update state in one place only if possible.
- In interviews, simple and correct beats clever and compressed.

Default Fallbacks

- `HashMap / Set` : best first guess when you want `O(1)` average lookup, counting, grouping, or deduplication.
- `Sorting` : best first guess when you want order, adjacency, intervals, binary search, or two pointers.

- Interview note: sorting is a standard fallback, but Python `list.sort()` is not strictly $O(1)$ extra space.

HashMap / Set

Recognition

- Need fast lookup, counting, deduplication, or grouping.
- You keep asking "have I seen this before?"

Quant / AI Relevance

High. This shows up in feature counting, dictionary joins, deduplication, token counting, caching, and frequency-based ranking.

Core Idea

Store the repeated lookup instead of recomputing it. Hash-based structures let you turn many nested-loop problems into one pass.

Python Template

```
def two_sum(nums, target):
    seen = {} # value -> index

    for i, x in enumerate(nums):
        need = target - x
        if need in seen:
            return [seen[need], i]
        seen[x] = i

    return []
```

Typical Complexity

- Time: $O(n)$
- Space: $O(n)$

Common Mistakes

- Writing the current value before checking its complement.
- Using a list for membership checks.
- Counting when a set is enough.

One-Line Memory Rule

If the same lookup happens again and again, store it in a `dict` or `set`.

Two Pointers

Recognition

- Sorted array, pair sum, palindrome, partition, or in-place compaction.
- One pointer move can safely eliminate part of the search space.

Quant / AI Relevance

High. Common in sorted numeric arrays, deduping, merging signals, and memory-efficient scans.

Core Idea

Two pointers replace nested loops when order lets you rule out candidates from one side at a time.

Python Template

```
def two_sum_sorted(nums, target):
    left, right = 0, len(nums) - 1

    while left < right:
        total = nums[left] + nums[right]

        if total == target:
            return [left, right]
        if total < target:
            left += 1
        else:
            right -= 1

    return [-1, -1]
```

Typical Complexity

- Time: $O(n)$
- Space: $O(1)$

Common Mistakes

- Using it on unsorted data without sorting or justifying order.
- Moving both pointers when only one should move.
- Forgetting duplicate skipping in problems like `3Sum`.

One-Line Memory Rule

If order lets you discard one side at a time, use two pointers.

Sliding Window

Recognition

- Subarray or substring with a local validity rule.
- Need longest, shortest, or count of a valid contiguous range.

Quant / AI Relevance

High. Very common in rolling constraints, time-series windows, token windows, and stream-style processing.

Core Idea

Keep a window `[left, right]` and update it incrementally. The key is to define what "valid" means and restore validity when it breaks.

Python Template

```
from collections import defaultdict

def length_of_longest_substring(s):
    freq = defaultdict(int)
    left = 0
    ans = 0

    for right, ch in enumerate(s):
        freq[ch] += 1 # include s[right]

        # shrink until the window is valid again
        while freq[ch] > 1:
            freq[s[left]] -= 1
            left += 1

        ans = max(ans, right - left + 1)

    return ans
```

Typical Complexity

- Time: $O(n)$
- Space: $O(k)$ for window state

Common Mistakes

- Using sliding window for exact-sum problems with negative numbers.
- Updating the answer before the window becomes valid again.
- Not writing down the window invariant.

One-Line Memory Rule

Window problems are about maintaining validity, not recomputing it.

Prefix Sum + HashMap

Recognition

- Exact subarray sum, count of subarrays, or range-total queries.
- Negative numbers exist, so shrinking a window is not reliable.

Quant / AI Relevance

High. Useful for cumulative signals, running totals, profit-style deltas, and exact-range statistics.

Core Idea

Turn a subarray condition into a difference between two prefix sums. Then store earlier prefix sums in a hashmap.

Python Template

```
from collections import defaultdict

def subarray_sum(nums, k):
    count = defaultdict(int)
    count[0] = 1 # empty prefix
    prefix = 0
    ans = 0

    for x in nums:
        prefix += x
        ans += count[prefix - k]
        count[prefix] += 1

    return ans
```

Typical Complexity

- Time: $O(n)$
- Space: $O(n)$

Common Mistakes

- Forgetting `count[0] = 1`.
- Using sliding window when negatives break monotonic behavior.
- Mixing exact totals with at-most constraints.

One-Line Memory Rule

Exact subarray total usually becomes prefix difference plus hashmap.

Stack

Recognition

- Nested structure, undo-last, iterative DFS, or recursion is banned.
- The most recent unfinished work should be handled first.

Quant / AI Relevance

Medium. Less central than arrays or heaps, but useful for parsers, iterative graph/tree traversal, and expression handling.

Core Idea

A stack is manual recursion. Push work when you discover it, pop work when you are ready to process it.

Python Template

```
def dfs_iterative(graph, start):
    stack = [start]
    seen = {start}
    order = []

    while stack:
        node = stack.pop()
        order.append(node)

        for nei in reversed(graph[node]):
            if nei not in seen:
                seen.add(nei)
                stack.append(nei)

    return order
```

Typical Complexity

- Time: $O(V + E)$
- Space: $O(V)$

Common Mistakes

- Forgetting that iterative DFS still needs `seen`.
- Using stack when a counter would be enough.
- Not recognizing that recursive DFS can be translated almost line by line.

One-Line Memory Rule

If recursion is the idea but recursion is inconvenient, use a stack.

Monotonic Stack

Recognition

- Next greater/smaller, previous greater/smaller, span, or histogram boundary.
- You need the nearest item with a monotonic relationship.

Quant / AI Relevance

Medium. Useful for nearest-threshold logic and boundary-finding patterns, especially on ordered arrays.

Core Idea

Maintain a stack that stays increasing or decreasing. When a new value breaks the rule, pop until the rule is restored.

Python Template

```
def next_greater_elements(nums):
    res = [-1] * len(nums)
    stack = [] # indices; values on stack are decreasing

    for i, x in enumerate(nums):
        while stack and nums[stack[-1]] < x:
            j = stack.pop()
            res[j] = x
        stack.append(i)

    return res
```

Typical Complexity

- Time: $O(n)$
- Space: $O(n)$

Common Mistakes

- Choosing the wrong monotonic direction.
- Storing values when indices are needed.
- Forgetting to ask whether you need next/previous and greater/smaller.

One-Line Memory Rule

Nearest greater or smaller usually means monotonic stack.

Binary Search

Recognition

- Sorted input, sorted answer space, or first/last valid boundary.
- The condition is monotonic: once true, it stays true.

Quant / AI Relevance

High. Common in threshold search, parameter tuning, feasibility checks, and ranked numeric arrays.

Core Idea

Binary search is really boundary search. You are not searching values; you are searching where the answer changes.

Python Template

```
def first_true(lo, hi, check):
    # smallest x in [lo, hi] such that check(x) is True
    while lo < hi:
        mid = (lo + hi) // 2

        if check(mid):
            hi = mid
        else:
            lo = mid + 1

    return lo
```

Typical Complexity

- Time: $O(\log n)$ for direct search, or $O(\log \text{ range} * \text{check_cost})$ on answers
- Space: $O(1)$

Common Mistakes

- No monotonic condition.
- Infinite loops from wrong boundary updates.
- Confusing answer search with index search.

One-Line Memory Rule

Binary search is about a monotonic boundary, not just sorted arrays.

Intervals

Recognition

- Start/end ranges, overlap checks, schedules, or merge decisions.
- Endpoint order matters more than original input order.

Quant / AI Relevance

Medium. Relevant for scheduling, time ranges, booking windows, and event sweeps.

Core Idea

Sort intervals first. Then sweep once while merging or counting overlaps.

Python Template

```
def merge(intervals):
    intervals.sort(key=lambda x: x[0])
    merged = []

    for start, end in intervals:
        if not merged or merged[-1][1] < start:
            merged.append([start, end])
        else:
            merged[-1][1] = max(merged[-1][1], end)

    return merged
```

Typical Complexity

- Time: $O(n \log n)$
- Space: $O(n)$ for output

Common Mistakes

- Forgetting to sort first.
- Getting the overlap condition wrong for touching intervals.
- Using nested loops before trying sort + sweep.

One-Line Memory Rule

Intervals usually mean sort, then sweep.

Heap / Priority Queue

Recognition

- Need repeated access to the current smallest/largest item.
- Need top k , least k , best-first search, or streaming rank maintenance.

Quant / AI Relevance

High. Common in ranking, retrieval, scheduling, leaderboards, beam-search-like thinking, and streaming analytics.

Core Idea

Heaps are for repeated best-item access. If you keep asking "what is the top item right now?", a heap is usually right.

Python Template

```
import heapq

def top_k_largest(nums, k):
    heap = []

    for x in nums:
        if len(heap) < k:
            heapq.heappush(heap, x)
        elif x > heap[0]:
            heapq.heapreplace(heap, x)

    return sorted(heap, reverse=True)
```

Typical Complexity

- Time: $O(n \log k)$
- Space: $O(k)$

Common Mistakes

- Forgetting Python `heapq` is a min-heap.
- Using a heap when you only need one kth answer once.
- Missing bucket sort as an option when frequencies are bounded.

One-Line Memory Rule

If you keep asking "what is the best item right now?", use a heap.

QuickSelect

Recognition

- Need one kth smallest/largest answer from static data.
- Full sorting feels wasteful.

Quant / AI Relevance

Medium. Good for one-shot rank selection, less useful than heaps for streaming or repeated top-k queries.

Core Idea

Partition like quicksort, but recurse only into the side containing the target index.

Python Template

```
import random

def find_kth_largest(nums, k):
    target = len(nums) - k

    def partition(left, right, pivot_index):
        pivot = nums[pivot_index]
        nums[pivot_index], nums[right] = nums[right], nums[pivot_index]
        store = left

        for i in range(left, right):
            if nums[i] < pivot:
                nums[store], nums[i] = nums[i], nums[store]
                store += 1

        nums[store], nums[right] = nums[right], nums[store]
        return store

    left, right = 0, len(nums) - 1
    while left <= right:
        pivot_index = random.randint(left, right)
        pivot_index = partition(left, right, pivot_index)

        if pivot_index == target:
            return nums[pivot_index]
        if pivot_index < target:
            left = pivot_index + 1
        else:
            right = pivot_index - 1
```

Typical Complexity

- Average Time: $O(n)$
- Worst Time: $O(n^2)$
- Space: $O(1)$ extra

Common Mistakes

- Forgetting kth-largest vs kth-smallest index conversion.
- Using it when you need stable order or repeated queries.
- Ignoring worst-case behavior in explanation.

One-Line Memory Rule

One kth-element query often means quickselect, not heap.

In-Place Array Tricks

Recognition

- Must use $O(1)$ extra space on an array.
- Values can be swapped to their "home" index or used to encode state.

Quant / AI Relevance

Lower, but still worth knowing because interviewers sometimes use it to test careful index reasoning.

Core Idea

Reuse the input array itself. Put values where they belong, reverse sections, or encode metadata using signs or offsets.

Python Template

```
def cyclic_sort(nums):
    i = 0

    while i < len(nums):
        j = nums[i] - 1

        if 1 <= nums[i] <= len(nums) and nums[i] != nums[j]:
            nums[i], nums[j] = nums[j], nums[i]
        else:
            i += 1

    return nums
```

Typical Complexity

- Time: $O(n)$
- Space: $O(1)$

Common Mistakes

- Infinite swap loops from bad guard conditions.
- Forgetting values may be out of range.
- Overwriting information you still need later.

One-Line Memory Rule

In-place array problems often mean "put each value where it belongs."

Linked List

Recognition

- Pointer wiring is the hard part.
- Head deletion, cycle detection, middle finding, merge, or reversal appears.

Quant / AI Relevance

Lower than arrays and heaps for these roles, but still standard interview material.

Core Idea

Simplify edge cases with a dummy node, fast/slow pointers, or iterative reversal.

Python Template

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def reverse_list(head):
    prev = None
    curr = head

    while curr:
        nxt = curr.next
        curr.next = prev
        prev = curr
        curr = nxt

    return prev
```

Typical Complexity

- Time: $O(n)$
- Space: $O(1)$

Common Mistakes

- Losing `next` before rewiring.
- Forgetting a dummy node when the head may change.
- Comparing values when node identity matters.

One-Line Memory Rule

Most linked list bugs disappear if you save `next` and use a dummy head.

Tree DFS / BFS

Recognition

- The input is a tree and each child subtree matters.
- You need depth, path info, levels, or subtree-combined answers.

Quant / AI Relevance

Medium. Trees are less central than arrays for quant, but still common for traversal and recursive reasoning.

Core Idea

Use DFS when each node needs information from children. Use BFS when you care about levels or shortest unweighted distance.

Python Template

```
from collections import deque

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def max_depth(root):
    if not root:
        return 0
    return 1 + max(max_depth(root.left), max_depth(root.right))

def level_order(root):
    if not root:
        return []

    q = deque([root])
    ans = []

    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
        ans.append(level)

    return ans
```

Typical Complexity

- Time: $O(n)$
- Space: $O(h)$ recursion for DFS, $O(w)$ queue for BFS

Common Mistakes

- Mixing "what the subtree returns" with "what the final answer stores".
- Forgetting base cases.
- Using BFS when a subtree recurrence is simpler.

One-Line Memory Rule

DFS solves subtree logic; BFS solves level logic.

Graph / Grid DFS/BFS

Recognition

- Need traversal, component counting, reachability, or unweighted shortest path.
- The state is a node, cell, coordinate, or string state with neighbors.

Quant / AI Relevance

High. Useful for search spaces, state machines, dependency graphs, pipeline reasoning, and shortest-step problems.

Core Idea

Model states as nodes and legal moves as edges. Use DFS for exploration and BFS for shortest unweighted steps.

Python Template

```
from collections import defaultdict, deque

def bfs_distances(n, edges, start):
    graph = defaultdict(list)
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)

    dist = {start: 0}
    q = deque([start])

    while q:
        node = q.popleft()

        for nei in graph[node]:
            if nei not in dist:
                dist[nei] = dist[node] + 1
                q.append(nei)

    return dist
```

Typical Complexity

- Time: $O(V + E)$
- Space: $O(V + E)$

Common Mistakes

- Marking visited too late and enqueueing duplicates.
- Using DFS when the question asks for shortest unweighted distance.
- Forgetting the graph may be disconnected.

One-Line Memory Rule

Graph problems start with nodes, neighbors, visited, and whether distance matters.

Topological Sort

Recognition

- Directed dependencies: course order, build order, prerequisite order, DAG pipeline order.
- Need an ordering that respects edge direction.

Quant / AI Relevance

High. Very relevant for DAG workflows, data pipelines, model dependencies, and scheduling constraints.

Core Idea

Nodes with indegree 0 are available now. Remove them layer by layer.

Python Template

```
from collections import defaultdict, deque

def topo_sort(num_nodes, edges):
    graph = defaultdict(list)
    indegree = [0] * num_nodes

    for u, v in edges:
        graph[u].append(v)
        indegree[v] += 1

    q = deque([i for i in range(num_nodes) if indegree[i] == 0])
    order = []

    while q:
        node = q.popleft()
        order.append(node)

        for nei in graph[node]:
            indegree[nei] -= 1
            if indegree[nei] == 0:
                q.append(nei)

    return order if len(order) == num_nodes else []
```

Typical Complexity

- Time: $O(V + E)$
- Space: $O(V + E)$

Common Mistakes

- Using it on undirected graphs.
- Reversing edge direction accidentally.
- Forgetting that a short result means there was a cycle.

One-Line Memory Rule

Dependencies in a DAG usually mean indegree queue.

Dijkstra

Recognition

- Weighted shortest path with non-negative edge costs.
- You want the cheapest path, not just any path.

Quant / AI Relevance

Medium. Useful for weighted search spaces, routing, and cost-based traversal.

Core Idea

Always expand the currently cheapest reachable state first with a min-heap.

Python Template

```
import heapq
from collections import defaultdict

def dijkstra(n, edges, start):
    graph = defaultdict(list)
    for u, v, w in edges:
        graph[u].append((v, w))

    dist = [float('inf')] * n
    dist[start] = 0
    heap = [(0, start)]

    while heap:
        curr_dist, node = heapq.heappop(heap)
        if curr_dist > dist[node]:
            continue

        for nei, weight in graph[node]:
            new_dist = curr_dist + weight
            if new_dist < dist[nei]:
                dist[nei] = new_dist
                heapq.heappush(heap, (new_dist, nei))

    return dist
```

Typical Complexity

- Time: $O((V + E) \log V)$
- Space: $O(V + E)$

Common Mistakes

- Using BFS on weighted edges.
- Forgetting stale heap entry checks.
- Using Dijkstra when negative weights exist.

One-Line Memory Rule

Non-negative weighted shortest path means Dijkstra.

Union Find

Recognition

- Repeated connectivity checks with merges.
- Components change over time and you only care who is connected.

Quant / AI Relevance

Lower to medium. Useful for connectivity and clustering-style reasoning, but usually less frequent than heaps or binary search.

Core Idea

Store a representative for each component, compress paths during finds, and merge by rank or size.

Python Template

```
parent = list(range(n))
rank = [0] * n

def find(x):
    if parent[x] != x:
        parent[x] = find(parent[x])
    return parent[x]

def union(a, b):
    ra, rb = find(a), find(b)
    if ra == rb:
        return False

    if rank[ra] < rank[rb]:
        parent[ra] = rb
    elif rank[ra] > rank[rb]:
        parent[rb] = ra
    else:
        parent[rb] = ra
        rank[ra] += 1

    return True
```

Typical Complexity

- Time: near $O(1)$ amortized per operation, more precisely $O(\alpha(n))$
- Space: $O(n)$

Common Mistakes

- Forgetting path compression.
- Recomputing connectivity from scratch every time.
- Using union find when you really need directed order, not connectivity.

One-Line Memory Rule

Repeated merge-and-query connectivity suggests union find.

Backtracking

Recognition

- Need all subsets, permutations, combinations, or valid constructions.
- The answer is built one choice at a time.

Quant / AI Relevance

Lower for most quant / AI interviews, but still good for search-space reasoning and pruning.

Core Idea

Choose, recurse, undo. The recursion tree is the search space.

Python Template

```
def subsets(nums):
    ans = []
    path = []

    def dfs(i):
        if i == len(nums):
            ans.append(path[:])
            return

        path.append(nums[i])
        dfs(i + 1)
        path.pop()

        dfs(i + 1)

    dfs(0)
    return ans
```

Typical Complexity

- Time: often exponential, commonly $O(2^n)$ or $O(n!)$
- Space: recursion depth plus output

Common Mistakes

- Forgetting to copy the current path.
- Forgetting to undo the last choice.
- Missing pruning opportunities.

One-Line Memory Rule

Backtracking is choose, recurse, unchoose.

Dynamic Programming

Recognition

- Best answer over many choices with repeated subproblems.
- A brute-force recursion would recompute the same state again and again.

Quant / AI Relevance

High. Important for optimization under constraints, sequence reasoning, and turning brute force into structured state transitions.

Core Idea

Define the state, the transition, and the base case. Then compute each state once.

Python Template

```
def coin_change(coins, amount):
    dp = [amount + 1] * (amount + 1)
    dp[0] = 0

    for total in range(1, amount + 1):
        for coin in coins:
            if total - coin >= 0:
                dp[total] = min(dp[total], dp[total - coin] + 1)

    return dp[amount] if dp[amount] != amount + 1 else -1
```

Typical Complexity

- Time: number of states times transitions
- Space: number of stored states

Common Mistakes

- State definition is unclear.
- Writing code before the recurrence is clear.
- Wrong iteration order when compressing to 1D.

One-Line Memory Rule

DP is just cached recursion with a clean state definition.

Greedy

Recognition

- A local choice looks safe and never needs to be undone.
- You can argue that one move always keeps you at least as good as alternatives.

Quant / AI Relevance

High. Very common in scheduling, resource allocation, interval selection, and one-pass optimization.

Core Idea

Make the best immediate move only if you can justify that it cannot block the global optimum.

Python Template

```
def can_jump(nums):
    farthest = 0

    for i, jump in enumerate(nums):
        if i > farthest:
            return False
        farthest = max(farthest, i + jump)

    return True
```

Typical Complexity

- Time: often $O(n)$ after sorting if needed
- Space: often $O(1)$

Common Mistakes

- Calling something greedy without proof intuition.
- Ignoring a small counterexample.
- Using greedy when DP is needed to compare multiple futures.

One-Line Memory Rule

Greedy works only when the local best move is globally safe.

Trie

Recognition

- Need prefix lookup, autocomplete, or prefix pruning.
- Many strings share prefixes and repeated prefix checks are expensive.

Quant / AI Relevance

Medium. More relevant for search, retrieval, autocomplete, and NLP-flavored prefix tasks than for typical quant problems.

Core Idea

Store characters along paths so shared prefixes are reused.

Python Template

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_word = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_word = True

    def search(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return node.is_word

    def starts_with(self, prefix):
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return True
```

Typical Complexity

- Insert/Search/Prefix Check: $O(\text{length of word})$
- Space: $O(\text{total characters stored})$

Common Mistakes

- Using a trie when a hashmap is enough.
- Forgetting the end-of-word marker.
- Building a trie when constraints are too small to justify it.

One-Line Memory Rule

Exact string lookup uses maps; shared prefix lookup suggests a trie.

Design Problems

Recognition

- You must implement a class with required operation complexity.
- The main challenge is choosing the right underlying data structures.

Quant / AI Relevance

High. This maps well to stream processing, temporal lookup, caches, serving layers, and data-system style questions.

Core Idea

Map each API call to primitives like array, hashmap, stack, heap, deque, linked list, trie, or binary search.

Python Template

```
from bisect import bisect_right
from collections import defaultdict

class TimeMap:
    def __init__(self):
        self.store = defaultdict(list) # key -> [(timestamp, value)]

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.store[key].append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        arr = self.store[key]
        i = bisect_right(arr, (timestamp, chr(127))) - 1
        return arr[i][1] if i >= 0 else ""
```

Typical Complexity

- set: $O(1)$ amortized
- get: $O(\log n)$ per key
- Space: $O(\text{total entries})$

Common Mistakes

- Ignoring the required complexity of each operation.
- Picking the right behavior with the wrong internal structure.
- Forgetting that design problems are usually combinations of simpler patterns.

One-Line Memory Rule

Design problems are mostly data-structure matching problems.

Mistake Log

Sliding Window vs Prefix Sum

- Use sliding window for local window validity you can maintain incrementally.
- Use prefix sum for exact cumulative totals, especially when negatives exist.
- Memory rule: window for validity, prefix for exact total.

Stack vs Counter

- Use stack when order or nesting matters.
- Use counter when only frequencies matter.
- Memory rule: nesting needs order, counts do not.

BFS vs DFS for Shortest Path

- BFS gives shortest path in an unweighted graph.
- DFS explores deeply but does not guarantee shortest path.
- Memory rule: shortest unweighted path means BFS.

Binary Search vs Two Pointers

- Binary search needs a monotonic condition.
- Two pointers need a direct move rule based on the current pair or window.
- Memory rule: boundary search vs moving scan.

DP vs Greedy

- Use DP when multiple futures must be compared.
- Use greedy only when the local move is provably safe.
- Memory rule: if the proof feels weak, it is probably not greedy.

Heap vs QuickSelect

- Heap is better for repeated top- `k` maintenance or streaming data.
- Quickselect is better for one static `kth` query.
- Memory rule: repeated extraction means heap; one-shot selection means quickselect.

Tree DFS Return Value vs Global Answer

- Return subtree information upward.
- Store the final cross-subtree answer separately when needed.
- Memory rule: ask whether the parent needs a value or the whole problem needs an answer.

Final Review Rules

- Classify the pattern before thinking about a problem number.
- For quant / AI prep, over-practice arrays, heaps, binary search, graphs, DP, greedy, and design.
- Write the invariant first if the pattern is not obvious.
- If stuck, ask which operation must be cheap: lookup, merge, connect, order, rank, or optimize.
- If still stuck, default to `HashMap` / `Set` or sorting to expose structure.